Stack, Queue & Priority Queue Templates

Four data structures — each with a canonical template and representative problems.

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
20	Valid Parentheses	Given a string of brackets `()[]{}`, return true if all brackets are properly closed and ordered.	the most recent unmatched open bracket is the only one a closing bracket can legally match — that is exactly LIFO.	O(n)	O(n)	Standard
155	Min Stack	Design a stack that supports push, pop, top, and `getMin()` in O(1).		O(1) per operation	O(n)	Standard
394	Decode String	Decode a string like `"3[a2[bc]]"` → `"abcbcabcbc"`.		O(n) (n = length of decoded output)	O(n)	Standard
739	Daily Temperatures	For each day, how many days until a warmer temperature? Return 0 if none.	a colder day just waits on the stack until the first warmer day arrives and resolves it.	O(n)	O(n)	Standard
496	Next Greater Element I	For each element in `nums1` (a subset of `nums2`), find the first greater element to its right in `nums2`. Return -1 if none.		O(m + n)	O(n)	Standard
84	Largest Rectangle in Histogram	Find the largest rectangle that can be formed within the histogram bars.	a bar can only stretch sideways until it hits a shorter bar; the stack remembers each bar's left limit so the right limit (current shorter bar) closes the rectangle.	O(n)	O(n)	Standard
42	Trapping Rain Water	How much water can be trapped between the bars after rain?	water sits in a dip between a left wall and a right wall, and its depth is set by whichever wall is shorter.	O(n)	O(n)	Standard
239	Sliding Window Maximum	Return the maximum in each sliding window of size k.	any element smaller than a newer element can never be the window max again, so discard it; the front always holds the current best.	O(n)	O(k)	Standard
1046	Last Stone Weight	Smash the two heaviest stones repeatedly. Return the last remaining weight (or 0).		O(n log n)	O(n)	Standard
347	Top K Frequent Elements	Return the k most frequent elements.		O(n log k)	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
295	Find Median from Data Stream	Add integers to a stream and find the median at any time in O(log n) add and O(1) find.	keep the smaller half and larger half balanced; the median always lives right at the boundary, on the tops of the two heaps.	O(log n) addNum, O(1) findMedian	O(n)	Standard
502	IPO	Before each project you must have enough capital. Start with capital `w`. Pick at most `k` projects to maximize final capital.		O(n log n)	O(n)	Standard
767	Reorganize String	Rearrange characters so no two adjacent characters are the same. Return `""` if impossible.		O(n log k) (k = distinct chars ≤ 26)	O(k)	Standard
503	Next Greater Element II	Given a circular integer array, return the next greater number for every element. The next greater number of an element is the first greater number found by traversing the array circularly.		O(n)	O(n)	Monotone decreasing stack. Iterate through the array TWICE (indices 0 to 2n-1, using `i % n`). Only push index `i` onto the stack during the first pass (`i < n`) to avoid duplicates.
85	Maximal Rectangle	Given a binary matrix filled with 0s and 1s, find the largest rectangle containing only 1s and return its area.		O(m·n)	O(n)	For each row, compute cumulative histogram heights (1s stack vertically). Then apply the Largest Rectangle in Histogram algorithm (#84) on each row's histogram.
224	Basic Calculator	Evaluate a string expression containing `+`, `-`, `(`, `)`, digits, and spaces.		O(n)	O(n)	When encountering `(`, push current (result, sign) onto stack and reset. When encountering `)`, combine current result with the saved result and sign from the stack.
227	Basic Calculator II	Evaluate a string expression with `+`, `-`, `*`, `/` and integers (no parentheses).		O(n)	O(n)	Process operator BEFORE the current number. Push positive/negative numbers for `+`/`-`. For `*`/`/`, immediately multiply/divide the top of the stack. Sum the stack at the end.
378	Kth Smallest Element in a Sorted Matrix	Given an n×n matrix where each row and column is sorted in ascending order, return the kth smallest element.		O(n log(max-min))	O(1)	Binary search on the VALUE range [matrix[0][0], matrix[n-1][n-1]]. For a given `mid`, count elements ≤ mid by scanning from the bottom-left corner: if `matrix[row][col] <= mid`, all `row+1` elements in this column (0..row) are ≤ mid.
373	Find K Pairs with Smallest Sums	Given two sorted arrays `nums1` and `nums2`, return the `k` pairs `(u, v)` (one from each) with the smallest sums.		O(k log k)	O(k)	Seed min-heap with `(nums1[i], nums2[0])` for i = 0..min(k, n1.length)-1. On each pop, if `j+1 < nums2.length`, push `(nums1[i], nums2[j+1])`. This expands row-by-row in the implicit pair matrix.
621	Task Scheduler	Given a list of tasks (characters) and a cooldown `n`, return the		O(k)	O(1)	Greedy formula. Find the most frequent task (maxFreq). It creates `maxFreq - 1` "frames" of `n +`

#	Name	Description	Intuition	Time	Space	Key Implementation
		minimum number of intervals to finish all tasks. Between two same tasks there must be at least `n` intervals.				`1` slots, plus `maxCount` (count of tasks with maxFreq) slots. The answer is `max(tasks.length, (maxFreq - 1) * (n + 1) + maxCount)`.
358	Rearrange String k Distance Apart	Rearrange a string so that the same characters are at least `k` distance apart. Return "" if impossible.		O(n log k)	O(k)	greedy with a max-heap by frequency plus a cooldown queue of size k that holds recently used characters until they're eligible again.
480	Sliding Window Median	Return the median of every window of size k as it slides across the array.		O(n·k) with heap remove (O(n log k) with indexed sets)	O(k)	two heaps (maxHeap = lower half, minHeap = upper half) with lazy deletion — defer removing elements that left the window, tracking balance with counts.
692	Top K Frequent Words	Return the k most frequent words. Sort by frequency descending; ties broken by lexicographic order.		O(n log k)	O(n)	min-heap of size k ordered so the "smallest" (lowest freq, or same freq with lexicographically larger word) sits on top for eviction.
772	Basic Calculator III	Evaluate an arithmetic expression with `+`, `-`, `*`, `/`, and parentheses.		O(n)	O(n)	combines #224 (parentheses via recursion) and #227 (operator precedence: apply `*`/`/` immediately, defer `+`/`-`).
32	Longest Valid Parentheses	Find the length of the longest valid (well-formed) parentheses substring.	push indices onto a stack; the bottom of the stack is always the index just before the current valid run, so the gap to it gives the run length.	O(n)	O(n)	Standard
71	Simplify Path	Simplify an absolute Unix file path (handle `.`, `..`, multiple slashes).	split on `/` and treat directories as a stack — `..` pops the last directory, `.` and empty tokens are ignored.	O(n)	O(n)	Standard
150	Evaluate Reverse Polish Notation	Evaluate a Reverse Polish Notation expression with `+`, `-`, `*`, `/`.	push operands; on an operator pop the two most recent operands, apply, and push the result back — exactly LIFO.	O(n)	O(n)	Standard
215	Kth Largest Element in an Array	Find the kth largest element in an unsorted array (not necessarily distinct).	a min-heap of size k keeps the k largest seen so far; its top is the kth largest.	O(n log k)	O(k)	Standard
218	The Skyline Problem	Given building rectangles, return the skyline as a list of key points.	sweep left to right over building edges; a max-heap of active heights tracks the tallest standing building, and a key point is emitted whenever that maximum changes.	O(n^2) (heap remove)	O(n)	Standard
225	Implement Stack using Queues	Implement a LIFO stack using only queue operations.	after each push, rotate the queue so the newest element sits at	O(n) push, O(1) others	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
			the front — then `poll` / `peek` behave like stack `pop` / `top` .			
232	Implement Queue using Stacks	Implement a queue using two stacks.	one stack receives pushes, the other serves pops; moving elements over reverses their order, restoring FIFO. Amortized O(1) per element.	O(1) amortized	O(n)	two stacks emulate FIFO
316	Remove Duplicate Letters	Remove duplicate letters so each appears once, result is lexicographically smallest.	monotone increasing stack of letters — pop a larger letter when a smaller one arrives, but only if the popped letter appears again later (so we can re-add it).	O(n)	O(1) (26 letters)	Standard
402	Remove K Digits	Remove k digits from a number string to get the smallest possible number.	monotone increasing stack of digits — whenever a smaller digit arrives, pop larger preceding digits (each pop is one removal) so high places hold the smallest digits.	O(n)	O(n)	Standard
456	132 Pattern	Check if a 1-3-2 pattern exists in an integer array.		O(n)	O(n)	scan right to left with a monotone decreasing stack; pop to track the largest value that is still smaller than some element to its right (the "2"). If a later element is below that "2", a 132 pattern exists.
506	Relative Ranks	Assign ranks ("Gold Medal", "Silver Medal", "Bronze Medal", then 4, 5, ...) to athletes by score.	a max-heap of (score, index) pops athletes in descending score order, so each pop assigns the next rank back to its original position.	O(n log n)	O(n)	Standard
636	Exclusive Time of Functions	Given a log of function start/end times, compute exclusive time per function.	a call stack mirrors execution; when a new call starts, the currently running function pauses (accumulate its slice), and when a call ends, the resumed parent restarts its clock.	O(L) (L = number of logs)	O(n)	Standard
678	Valid Parenthesis String	Check if a string with `(` , `)` , `*` (wildcard) can be a valid parenthesis string.		O(n)	O(1)	track a range `[low, high]` of possible open-paren counts; `*` widens the range (could be `(` , `)` , or empty). Valid iff the range can return to 0 and never goes negative.
703	Kth Largest Element in a Stream	Design a class to find the kth largest element in a stream.	a min-heap capped at size k always holds the k largest seen; its top is the running kth largest.	O(log k) per add	O(k)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
716	Max Stack	Design a stack with push, pop, top, getMax, and popMax operations.		O(n) popMax, O(1) others	O(n)	mirror an auxiliary `maxStack` (like #155). `popMax` pops down to the max into a temp buffer, removes it, then pushes the buffer back — re-establishing both stacks.
735	Asteroid Collision	Simulate asteroids moving in a row: right-moving collide with left-moving.	a stack holds surviving asteroids; a left-moving asteroid only collides with a right-moving one on top, resolving collisions repeatedly until it survives, explodes, or the stack clears.	O(n)	O(n)	Standard
759	Employee Free Time	Find the free time intervals common to all employees given their schedules.	flatten all intervals, sort by start, then sweep tracking the furthest end seen so far; any gap between that end and the next start is common free time.	O(n log n)	O(n)	Standard
844	Backspace String Compare	Check if two strings are equal after processing `#` as backspace.	build each string with a stack where `#` pops the last character, then compare the resulting stacks.	O(n)	O(n)	Standard
856	Score of Parentheses	Calculate the score of a valid parentheses string (empty=0, AB=A+B, (A)=2A or 1 if empty).		O(n)	O(n)	stack holds the accumulated score at each depth; push 0 on `(`, and on `)` collapse the inner score (`max(2*inner, 1)`) into the parent frame.
862	Shortest Subarray with Sum at Least K	Find the length of the shortest subarray with sum at least k (k can be negative).		O(n)	O(n)	monotone increasing deque over prefix sums — pop from the front when a window qualifies, and from the back when a newer prefix is smaller (dominating older larger ones).
907	Sum of Subarray Minimums	Find the sum of subarray minimums for all subarrays.		O(n)	O(n)	monotone increasing stack — for each element as the minimum, count subarrays via distance to the previous strictly-smaller and next smaller-or-equal element, then weight by the value.
921	Minimum Add to Make Parentheses Valid	Find the minimum additions to make a parentheses string valid.	track open parentheses as a counter (stack of size); each unmatched `)` needs an added `(`, and leftover `(` each need a `)`.	O(n)	O(1)	Standard
946	Validate Stack Sequences	Check if a sequence can be produced by a series of push-pop operations on a stack.	simulate — push each value, then greedily pop whenever the stack top matches the next expected popped value. If everything pops, the sequence is valid.	O(n)	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
973	K Closest Points to Origin	Find the k points closest to the origin.	a max-heap of size k keyed on squared distance keeps the k closest seen; evict the farthest whenever the heap overflows.	O(n log k)	O(k)	Standard
1086	High Five	For each student, compute the average of their top five scores.	keep a min-heap of size 5 per student so only their five highest scores remain, then average those.	O(n log 5)	O(n)	Standard
1190	Reverse Substrings Between Each Pair of Parentheses	Reverse the substrings between each pair of parentheses (innermost first).		O(n^2)	O(n)	stack of builders — push current builder on `(` ; on `)` reverse the inner builder and append it to the parent frame.
1209	Remove All Adjacent Duplicates in String II	Remove all adjacent duplicates of length k in a string repeatedly.		O(n)	O(n)	stack of (char, count) pairs — increment the top's count on a repeat, and pop the frame once its count reaches k.
1249	Minimum Remove to Make Valid Parentheses	Remove the minimum number of parentheses to make the string valid.		O(n)	O(n)	stack stores indices of unmatched `(` ; a `)` with no match is marked for removal, and any `(` left on the stack at the end is also removed.
1337	The K Weakest Rows in a Matrix	Find the k weakest rows in a matrix (fewest soldiers, ties broken by row index).		O(m·n + m log k)	O(k)	max-heap of size k keyed on (soldier count, row index) so the strongest qualifying row sits on top for eviction; the remaining k are the weakest.
1541	Minimum Insertions to Balance a Parentheses String	Find the minimum insertions to make a string balanced (every `(` needs two `)`).		O(n)	O(1)	counter-style stack tracking open `(` ; each `(` demands two `)` . Handle `)` carefully since they come in pairs, inserting a `)` when only one is available.
1614	Maximum Nesting Depth of the Parentheses	Find the maximum nesting depth of parentheses in a string.	the stack depth equals the current nesting level; track a running counter for `(` / `)` and record its peak.	O(n)	O(1)	Standard
1792	Maximum Average Pass Ratio	Maximize the average pass ratio by adding extra students optimally.		O((n + extra) log n)	O(n)	max-heap keyed on the marginal gain from adding one student to a class; greedily assign each extra student where it helps most, then push the updated gain back.
1944	Number of Visible People in a Queue	Count the number of people each person can see in a queue (taller blocks view).		O(n)	O(n)	monotone decreasing stack scanned right to left; pop shorter people (each is visible) until a taller one blocks the view — that taller person is visible too.
1985	Find the Kth Largest Integer in the Array	Find the kth largest integer in an array of number strings.		O(n log k)	O(k)	min-heap of size k comparing the numeric strings by length first, then lexicographically (so big-integer values compare correctly without overflow).

All Templates

```
// 1. STACK (LIFO) — use ArrayDeque, not Stack class
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x);     // add to top
stack.pop();       // remove from top
stack.peek();      // view top, no remove

// 2A. MONOTONE DECREASING STACK — next GREATER element
// MENTAL MODEL: keep only candidates still waiting for a bigger neighbor; current resolves them all.
// WHEN: "next/previous greater element", "warmer/taller to the right"
// Invariant: stack values decrease from bottom to top
// Pop when current is GREATER than top → top found its next greater
Deque<Integer> stack = new ArrayDeque<>();  // stores indices
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] < nums[i])
        result[stack.pop()] = nums[i];     // nums[i] is next greater for popped index
    stack.push(i);
}
// Remaining in stack: no next greater element exists

// 2B. MONOTONE INCREASING STACK — next SMALLER element / span width
// MENTAL MODEL: each popped bar's reach extends until something shorter blocks it on both sides.
// WHEN: "largest rectangle", "trapped water", "span/width bounded by smaller elements"
// Invariant: stack values increase from bottom to top
// Pop when current is SMALLER than top → use popped index to compute width/span
Deque<Integer> stack = new ArrayDeque<>();  // stores indices
for (int i = 0; i < n; i++) {
    while (!stack.isEmpty() && nums[stack.peek()] > nums[i]) {
        int height = nums[stack.pop()];
        int width  = stack.isEmpty() ? i : i - stack.peek() - 1;  // span between boundaries
        // process height * width
    }
    stack.push(i);
}

// 3. MONOTONE DEQUE — sliding window max/min
// MENTAL MODEL: the front is the current window's answer; anything a newer-and-bigger value beats is d
// WHEN: "max/min of every window of size k"
// Deque stores INDICES; values at those indices are decreasing front-to-back (for max)
Deque<Integer> queue = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
    while (!queue.isEmpty() && nums[queue.peekLast()] <= nums[i])
        queue.pollLast();               // remove smaller elements — they can never be window max
    queue.offerLast(i);
    if (queue.peekFirst() < i - k + 1) queue.pollFirst();  // evict out-of-window elements
    if (i >= k - 1) result[i - k + 1] = nums[queue.peekFirst()];
}

// 4. PRIORITY QUEUE
PriorityQueue<Integer> minPQ = new PriorityQueue<>();                      // min-heap (default)
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
PriorityQueue<int[]>    pq    = new PriorityQueue<>((a, b) -> a[1] - b[1]); // custom: sort by index 1
pq.offer(x);   // add
pq.poll();     // remove and return min/max
pq.peek();     // view min/max without removing

// 5. TWO HEAPS — maintain median in O(log n)
// MENTAL MODEL: split the data at the median; the median is always sitting on the two heaps' tops.
// WHEN: "running median", "median of a data stream"
// maxHeap = lower half (smaller numbers), minHeap = upper half (larger numbers)
// Invariant: maxHeap.size() == minHeap.size() or maxHeap.size() == minHeap.size() + 1
```

```java
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
void addNum(int num) {
    maxHeap.offer(num);
    minHeap.offer(maxHeap.poll());              // push one to minHeap (possibly num itself)
    if (maxHeap.size() < minHeap.size())
        maxHeap.offer(minHeap.poll());          // rebalance: maxHeap always ≥ minHeap in size
}
double findMedian() {
    return maxHeap.size() > minHeap.size()
        ? maxHeap.peek()
        : (maxHeap.peek() + minHeap.peek()) / 2.0;
}
```

Part 1 — Basic Stack

#20 Valid Parentheses

Description: Given a string of brackets `()[]{}`, return true if all brackets are properly closed and ordered.

Intuition: the most recent unmatched open bracket is the only one a closing bracket can legally match — that is exactly LIFO.

```java
class Solution {
    public boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') {
                stack.push(c);
            } else {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if (c == ')' && top != '(') return false;
                if (c == ']' && top != '[') return false;
                if (c == '}' && top != '{') return false;
            }
        }
        return stack.isEmpty();
    }
}
```

Time O(n) | **Space** O(n)

#155 Min Stack

Description: Design a stack that supports push, pop, top, and `getMin()` in O(1).
Key: auxiliary `minStack` mirrors the main stack — each level stores the current running minimum.

```java
class MinStack {
    Deque<Integer> stack    = new ArrayDeque<>();
    Deque<Integer> minStack = new ArrayDeque<>();   // ← always push current min alongside value

    public void push(int val) {
        stack.push(val);
        minStack.push(minStack.isEmpty() ? val : Math.min(minStack.peek(), val));
    }
    public void pop()    { stack.pop(); minStack.pop(); }  // ← pop both in sync
    public int top()     { return stack.peek(); }
    public int getMin()  { return minStack.peek(); }
}
```

Time O(1) per operation | **Space** O(n)

#394 Decode String

Description: Decode a string like `"3[a2[bc]]"` → `"abcbcabcbc"`.
Key: two stacks — one for repeat counts, one for the string built before each `[`. On `]`, pop both and repeat.

```java
class Solution {
    public String decodeString(String s) {
        Deque<Integer>      countStack = new ArrayDeque<>();
        Deque<StringBuilder> strStack  = new ArrayDeque<>();
        StringBuilder current = new StringBuilder();
        int k = 0;
        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) {
                k = k * 10 + (c - '0');                          // ← multi-digit number
            } else if (c == '[') {
                countStack.push(k);                               // ← save count
                strStack.push(current);                           // ← save string so far
                current = new StringBuilder();
                k = 0;
            } else if (c == ']') {
                int count = countStack.pop();
                StringBuilder parent = strStack.pop();
                for (int i = 0; i < count; i++) parent.append(current);  // ← repeat
                current = parent;
            } else {
                current.append(c);
            }
        }
        return current.toString();
    }
}
```

Time O(n) (n = length of decoded output) | **Space** O(n)

Part 2 — Monotone Stack

Decreasing vs Increasing

```
DECREASING stack (pop when current > top):     finds NEXT GREATER element for each popped index
INCREASING stack (pop when current < top):     finds NEXT SMALLER element — used for span width

Both store INDICES (not values) so you can compute distances and widths.
```

#739 Daily Temperatures

Description: For each day, how many days until a warmer temperature? Return 0 if none.
Template: monotone decreasing — when current temp is greater, pop and record the gap.

Intuition: a colder day just waits on the stack until the first warmer day arrives and resolves it.

```java
class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] result = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();   // indices; temps decrease from bottom to top
        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && temperatures[stack.peek()] < temperatures[i]) {
                int idx = stack.pop();
                result[idx] = i - idx;               // ← days until warmer = distance
            }
            stack.push(i);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#496 Next Greater Element I

Description: For each element in `nums1` (a subset of `nums2`), find the first greater element to its right in `nums2`. Return -1 if none.
Key: precompute NGE for all elements in `nums2` using a monotone stack + HashMap. Then look up each `nums1` element.

```java
class Solution {
    public int[] nextGreaterElement(int[] nums1, int[] nums2) {
        Map<Integer, Integer> nextGreater = new HashMap<>();
        Deque<Integer> stack = new ArrayDeque<>();
        for (int num : nums2) {
            while (!stack.isEmpty() && stack.peek() < num)
                nextGreater.put(stack.pop(), num);   // ← num is next greater for stack.peek()
            stack.push(num);
        }
        int[] result = new int[nums1.length];
        for (int i = 0; i < nums1.length; i++)
            result[i] = nextGreater.getOrDefault(nums1[i], -1);
        return result;
    }
}
```

Time O(m + n) | **Space** O(n)

#84 Largest Rectangle in Histogram

Description: Find the largest rectangle that can be formed within the histogram bars.
Template: monotone increasing stack (pop when current bar is shorter). Popped bar's width = from `stack.peek() + 1` to `i - 1`.
Key: add sentinel `0` s at both ends to flush all remaining bars from the stack.

Intuition: a bar can only stretch sideways until it hits a shorter bar; the stack remembers each bar's left limit so the right limit (current shorter bar) closes the rectangle.

```java
class Solution {
    public int largestRectangleArea(int[] heights) {
        int n = heights.length;
        int[] h = new int[n + 2];                          // ← sentinels: h[0]=h[n+1]=0
        System.arraycopy(heights, 0, h, 1, n);
        Deque<Integer> stack = new ArrayDeque<>();
        int maxArea = 0;
        for (int i = 0; i <= n + 1; i++) {
            while (!stack.isEmpty() && h[stack.peek()] > h[i]) {
                int height = h[stack.pop()];
                int left   = stack.isEmpty() ? -1 : stack.peek();  // left boundary
                int width  = i - left - 1;                 // bars strictly between left boundary and cu
                maxArea = Math.max(maxArea, height * width);
            }
            stack.push(i);
        }
        return maxArea;
    }
}
```

Time O(n) | **Space** O(n)

#42 Trapping Rain Water

Description: How much water can be trapped between the bars after rain?
Template: monotone increasing stack — when current bar is taller, water fills the "valley" between the current bar and the bar now at the top of the stack.

Intuition: water sits in a dip between a left wall and a right wall, and its depth is set by whichever wall is shorter.

```java
class Solution {
    public int trap(int[] height) {
        Deque<Integer> stack = new ArrayDeque<>();
        int water = 0;
        for (int i = 0; i < height.length; i++) {
            while (!stack.isEmpty() && height[stack.peek()] < height[i]) {
                int bottom   = height[stack.pop()];
                if (stack.isEmpty()) break;
                int left     = stack.peek();
                int h        = Math.min(height[left], height[i]) - bottom;  // bounded by the shorter w
                int w        = i - left - 1;
                water       += h * w;
            }
            stack.push(i);
        }
        return water;
    }
}
```

Time O(n) | **Space** O(n)

84 vs 42 — Side by Side

```
                    #84 Largest Rectangle       #42 Trapping Rain Water
Pop when:           h[current] < h[top]           h[current] > h[top]
Stack order:        increasing (bottom→top)      increasing (bottom→top)
Popped bar role:    height of rectangle          floor of the trapped water valley
Width from:         (i - stack.peek() - 1)       (i - stack.peek() - 1)
Area/water:         height × width               min(left_wall, right_wall) × width
```

Part 3 — Monotone Deque

#239 Sliding Window Maximum

Description: Return the maximum in each sliding window of size k.
Template: deque stores indices; values at those indices are strictly decreasing front-to-back (max at front).

Intuition: any element smaller than a newer element can never be the window max again, so discard it; the front always holds the current best.

```java
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] result = new int[n - k + 1];
        Deque<Integer> queue = new ArrayDeque<>();   // indices, values decrease front→back
        for (int i = 0; i < n; i++) {
            while (!queue.isEmpty() && nums[queue.peekLast()] <= nums[i])
                queue.pollLast();                    // ← remove smaller: they can never be max
            queue.offerLast(i);
            if (queue.peekFirst() < i - k + 1)
                queue.pollFirst();                   // ← evict index out of window
            if (i >= k - 1)
                result[i - k + 1] = nums[queue.peekFirst()];  // ← front = window max
        }
        return result;
    }
}
```

Time O(n) | **Space** O(k)

Part 4 — Priority Queue

#1046 Last Stone Weight

Description: Smash the two heaviest stones repeatedly. Return the last remaining weight (or 0).
Template: max-heap — always poll the two largest.

```java
class Solution {
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
        for (int stone : stones) pq.offer(stone);
        while (pq.size() > 1) {
            int a = pq.poll(), b = pq.poll();
            if (a != b) pq.offer(a - b);             // ← only add remainder if different
        }
        return pq.isEmpty() ? 0 : pq.poll();
    }
}
```

Time O(n log n) | **Space** O(n)

#347 Top K Frequent Elements

Description: Return the k most frequent elements.
Template: min-heap of size k — evict the least frequent when size exceeds k. What remains = top k.

```java
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int num : nums) freq.merge(num, 1, Integer::sum);
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);  // min-heap on freq
        for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
            pq.offer(new int[]{e.getKey(), e.getValue()});
            if (pq.size() > k) pq.poll();            // ← evict least frequent
        }
        return pq.stream().mapToInt(a -> a[0]).toArray();
    }
}
```

Time O(n log k) | **Space** O(n)

#295 Find Median from Data Stream

Description: Add integers to a stream and find the median at any time in O(log n) add and O(1) find.
Template: two heaps — `maxHeap` holds lower half, `minHeap` holds upper half. Always rebalance so `maxHeap.size() ≥ minHeap.size()`.

Intuition: keep the smaller half and larger half balanced; the median always lives right at the boundary, on the tops of the two heaps.

```java
class MedianFinder {
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder()); // lower half
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();                           // upper half

    public void addNum(int num) {
        maxHeap.offer(num);
        minHeap.offer(maxHeap.poll());               // ← always push one to minHeap first
        if (maxHeap.size() < minHeap.size())
            maxHeap.offer(minHeap.poll());           // ← rebalance: maxHeap always ≥ minHeap size
    }
    public double findMedian() {
        return maxHeap.size() > minHeap.size()
            ? maxHeap.peek()
            : (maxHeap.peek() + minHeap.peek()) / 2.0;
    }
}
```

Time O(log n) addNum, O(1) findMedian | **Space** O(n)

#502 IPO

Description: Before each project you must have enough capital. Start with capital `w`. Pick at most `k` projects to maximize final capital.
Template: sort by required capital + max-heap of profits. At each step, unlock all affordable projects (move them into the heap), then greedily pick the most profitable.

```java
class Solution {
    public int findMaximizedCapital(int k, int w, int[] profits, int[] capital) {
        int n = profits.length;
        int[][] projects = new int[n][2];
        for (int i = 0; i < n; i++) projects[i] = new int[]{capital[i], profits[i]};
        Arrays.sort(projects, (a, b) -> a[0] - b[0]);               // ← sort by required capital
        PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder()); // max-heap of pro
        int idx = 0;
        for (int i = 0; i < k; i++) {
            while (idx < n && projects[idx][0] <= w) pq.offer(projects[idx++][1]); // ← unlock
            if (pq.isEmpty()) break;
            w += pq.poll();                                          // ← pick most profitable
        }
        return w;
    }
}
```

Time O(n log n) | **Space** O(n)

#767 Reorganize String

Description: Rearrange characters so no two adjacent characters are the same. Return `""` if impossible.
Template: max-heap of (char, count) — at each step, pick the two most frequent chars and append them alternately.

```java
class Solution {
    public String reorganizeString(String s) {
        int[] count = new int[26];
        for (char c : s.toCharArray()) count[c - 'a']++;
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> b[1] - a[1]); // max-heap on count
        for (int i = 0; i < 26; i++) if (count[i] > 0) pq.offer(new int[]{i, count[i]});
        StringBuilder builder = new StringBuilder();
        while (pq.size() > 1) {
            int[] a = pq.poll(), b = pq.poll();
            builder.append((char)('a' + a[0]));
            builder.append((char)('a' + b[0]));
            if (--a[1] > 0) pq.offer(a);
            if (--b[1] > 0) pq.offer(b);
        }
        if (!pq.isEmpty()) {
            int[] last = pq.poll();
            if (last[1] > 1) return "";                          // ← only one char left, count > 1:
            builder.append((char)('a' + last[0]));
        }
        return builder.toString();
    }
}
```

Time O(n log k) (k = distinct chars ≤ 26) | **Space** O(k)

Structure Chooser

Goal	Structure	Why
Next greater/smaller element	Monotone stack	O(n) total — each element pushed/popped once
Sliding window max/min	Monotone deque	Front = window extreme; stale indices evicted
Global min/max, k-th element	PriorityQueue	O(log n) per op
Running median	Two heaps	Split at median; each heap half is O(log n)
O(1) stack minimum	Aux min-stack	Mirror min value at every level
Nested structure (brackets, k-repeats)	Two stacks	One for counts, one for strings

Deque API Cheat Sheet

One class, three roles. `ArrayDeque` adds/removes at *both* ends, so stack, queue, and deque are just usage patterns of the same structure — the only difference is **which end you remove from**. Always declare it `Deque<Integer> queue = new ArrayDeque<>();` (never the legacy `Stack` class, which is synchronized and `Vector`-backed; and `ArrayDeque` beats `LinkedList` as a queue). Name the variable for the *role* it plays (`stack` / `queue`).

```
        addFirst / push                         addLast / offer
              ↓                                       ↓
            [ FRONT ............................ BACK ]
              ↑                                       ↑
        pollFirst / pop                         pollLast

STACK (LIFO): add + remove at FRONT  → newest out first
QUEUE (FIFO): add at BACK, remove at FRONT → oldest out first
```

Intent	Stack (LIFO)	Queue (FIFO)
Add	`push(x)` → `addFirst`	`offer(x)` → `addLast`
Remove	`pop()` → `removeFirst`	`poll()` → `removeFirst`
Peek	`peek()` → `peekFirst`	`peek()` → `peekFirst`

Both **remove from the front** — the discipline comes from *where you add*: stack adds to the front (so newest is popped = LIFO), queue adds to the back (so oldest is polled = FIFO).

```java
Deque<Integer> queue = new ArrayDeque<>();

// As STACK (LIFO):    push()/pop()/peek()  → operate on FRONT
queue.push(x);    // = offerFirst
queue.pop();      // = pollFirst
queue.peek();     // = peekFirst

// As QUEUE (FIFO):   offer()/poll()/peek() → back-in, front-out
queue.offerLast(x);
queue.pollFirst();
queue.peekFirst();

// As DEQUE:          explicit first/last (e.g. monotone sliding-window)
queue.offerFirst(x);  queue.pollFirst();  queue.peekFirst();
queue.offerLast(x);   queue.pollLast();   queue.peekLast();
```

#503 Next Greater Element II

Description: Given a circular integer array, return the next greater number for every element. The next greater number of an element is the first greater number found by traversing the array circularly.

Variation: Monotone decreasing stack. Iterate through the array TWICE (indices 0 to 2n-1, using `i % n`). Only push index `i` onto the stack during the first pass (`i < n`) to avoid duplicates.

```
class Solution {
    public int[] nextGreaterElements(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        Arrays.fill(result, -1);
        Deque<Integer> stack = new ArrayDeque<>();   // monotone decreasing stack of indices
        for (int i = 0; i < 2 * n; i++) {           // ← VARIATION: iterate twice for circular
            while (!stack.isEmpty() && nums[stack.peek()] < nums[i % n])
                result[stack.pop()] = nums[i % n];
            if (i < n) stack.push(i);               // ← VARIATION: only push in first pass
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#85 Maximal Rectangle

Description: Given a binary matrix filled with 0s and 1s, find the largest rectangle containing only 1s and return its area.

Variation: For each row, compute cumulative histogram heights (1s stack vertically). Then apply the Largest Rectangle in Histogram algorithm (#84) on each row's histogram.

```java
class Solution {
    public int maximalRectangle(char[][] matrix) {
        int cols = matrix[0].length;
        int[] heights = new int[cols];
        int maxArea = 0;
        for (char[] row : matrix) {
            for (int j = 0; j < cols; j++)
                heights[j] = row[j] == '1' ? heights[j] + 1 : 0;  // ← VARIATION: build histogram row b
            maxArea = Math.max(maxArea, largestRectangle(heights));
        }
        return maxArea;
    }

    private int largestRectangle(int[] heights) {
        int n = heights.length;
        // Pad with 0s on both ends to flush remaining stack
        int[] h = new int[n + 2];
        System.arraycopy(heights, 0, h, 1, n);
        Deque<Integer> stack = new ArrayDeque<>();
        int max = 0;
        for (int i = 0; i <= n + 1; i++) {
            while (!stack.isEmpty() && h[stack.peek()] > h[i]) {
                int height = h[stack.pop()];
                max = Math.max(max, height * (i - stack.peek() - 1));
            }
            stack.push(i);
        }
        return max;
    }
}
```

Time O(m·n) | **Space** O(n)

#224 Basic Calculator

Description: Evaluate a string expression containing `+`, `-`, `(`, `)`, digits, and spaces.

Variation: When encountering `(`, push current (result, sign) onto stack and reset. When encountering `)`, combine current result with the saved result and sign from the stack.

```java
class Solution {
    public int calculate(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        int result = 0, number = 0, sign = 1;
        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) {
                number = number * 10 + (c - '0');
            } else if (c == '+') {
                result += sign * number; number = 0; sign = 1;
            } else if (c == '-') {
                result += sign * number; number = 0; sign = -1;
            } else if (c == '(') {
                stack.push(result);    // ← VARIATION: save result and sign before '('
                stack.push(sign);
                result = 0; sign = 1;
            } else if (c == ')') {
                result += sign * number; number = 0;
                result *= stack.pop();  // ← VARIATION: multiply by sign before '('
                result += stack.pop();  // ← VARIATION: add result before '('
            }
        }
        return result + sign * number;
    }
}
```

Time O(n) | **Space** O(n)

#227 Basic Calculator II

Description: Evaluate a string expression with `+`, `-`, `*`, `/` and integers (no parentheses).

Variation: Process operator BEFORE the current number. Push positive/negative numbers for `+`/`-`. For `*`/`/`, immediately multiply/divide the top of the stack. Sum the stack at the end.

```java
class Solution {
    public int calculate(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        int number = 0;
        char sign = '+';                                    // ← VARIATION: track previous operator
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (Character.isDigit(c)) number = number * 10 + (c - '0');
            if ((!Character.isDigit(c) && c != ' ') || i == s.length() - 1) {
                if      (sign == '+') stack.push(number);
                else if (sign == '-') stack.push(-number);
                else if (sign == '*') stack.push(stack.pop() * number);  // ← VARIATION: apply * immedi
                else if (sign == '/') stack.push(stack.pop() / number);  // ← VARIATION: apply / immedi
                sign = c; number = 0;
            }
        }
        return stack.stream().mapToInt(Integer::intValue).sum();
    }
}
```

Time O(n) | **Space** O(n)

#378 Kth Smallest Element in a Sorted Matrix

Description: Given an n×n matrix where each row and column is sorted in ascending order, return the kth smallest element.

Variation: Binary search on the VALUE range [matrix[0][0], matrix[n-1][n-1]]. For a given `mid`, count elements ≤ mid by scanning from the bottom-left corner: if `matrix[row][col] <= mid`, all `row+1` elements in this column (0..row) are ≤ mid.

```
class Solution {
    public int kthSmallest(int[][] matrix, int k) {
        int n = matrix.length;
        int i = matrix[0][0], j = matrix[n-1][n-1];        // ← VARIATION: binary search on value range
        while (i < j) {
            int mid = i + (j - i) / 2;
            if (countLE(matrix, mid, n) >= k) {
                j = mid;
            } else {
                i = mid + 1;
            }
        }
        return i;
    }

    private int countLE(int[][] matrix, int target, int n) {
        int count = 0, row = n - 1, col = 0;               // ← VARIATION: scan from bottom-left
        while (row >= 0 && col < n) {
            if (matrix[row][col] <= target) {
                count += row + 1;
                col++;
            } else {
                row--;
            }
        }
        return count;
    }
}
```

Time O(n log(max-min)) | **Space** O(1)

#373 Find K Pairs with Smallest Sums

Description: Given two sorted arrays `nums1` and `nums2`, return the `k` pairs `(u, v)` (one from each) with the smallest sums.

Variation: Seed min-heap with `(nums1[i], nums2[0])` for i = 0..min(k, n1.length)-1. On each pop, if `j+1 < nums2.length`, push `(nums1[i], nums2[j+1])`. This expands row-by-row in the implicit pair matrix.

```java
class Solution {
    public List<List<Integer>> kSmallestPairs(int[] nums1, int[] nums2, int k) {
        List<List<Integer>> result = new ArrayList<>();
        if (nums1.length == 0 || nums2.length == 0) return result;
        // heap stores [i, j] meaning pair (nums1[i], nums2[j])
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) ->
            nums1[a[0]] + nums2[a[1]] - nums1[b[0]] - nums2[b[1]]);
        for (int i = 0; i < Math.min(nums1.length, k); i++)
            pq.offer(new int[]{i, 0});              // ← VARIATION: seed with (i, 0) for each row
        while (!pq.isEmpty() && result.size() < k) {
            int[] current = pq.poll();
            result.add(Arrays.asList(nums1[current[0]], nums2[current[1]]));
            if (current[1] + 1 < nums2.length)
                pq.offer(new int[]{current[0], current[1] + 1});  // ← VARIATION: expand j in same row
        }
        return result;
    }
}
```

Time O(k log k) | **Space** O(k)

#621 Task Scheduler

Description: Given a list of tasks (characters) and a cooldown `n`, return the minimum number of intervals to finish all tasks. Between two same tasks there must be at least `n` intervals.

Variation: Greedy formula. Find the most frequent task (maxFreq). It creates `maxFreq - 1` "frames" of `n + 1` slots, plus `maxCount` (count of tasks with maxFreq) slots. The answer is `max(tasks.length, (maxFreq - 1) * (n + 1) + maxCount)`.

```java
class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] count = new int[26];
        for (char c : tasks) count[c - 'A']++;
        Arrays.sort(count);
        int maxFreq = count[25];
        int maxCount = 0;
        for (int c : count) if (c == maxFreq) maxCount++;          // ← VARIATION: count ties at max
        return Math.max(tasks.length, (maxFreq - 1) * (n + 1) + maxCount);  // ← VARIATION: formula
    }
}
```

Time O(k) | **Space** O(1)

#358 Rearrange String k Distance Apart

Description: Rearrange a string so that the same characters are at least `k` distance apart. Return "" if impossible.
Variation: greedy with a max-heap by frequency plus a cooldown queue of size k that holds recently used characters until they're eligible again.

```java
class Solution {
    public String rearrangeString(String s, int k) {
        if (k <= 1) return s;
        Map<Character, Integer> count = new HashMap<>();
        for (char c : s.toCharArray()) count.merge(c, 1, Integer::sum);
        PriorityQueue<Map.Entry<Character,Integer>> maxHeap =
            new PriorityQueue<>((a, b) -> b.getValue() - a.getValue());
        maxHeap.addAll(count.entrySet());
        Queue<Map.Entry<Character,Integer>> cooldown = new ArrayDeque<>();  // ← VARIATION: cooldown qu
        StringBuilder builder = new StringBuilder();
        while (!maxHeap.isEmpty()) {
            var entry = maxHeap.poll();
            builder.append(entry.getKey());
            entry.setValue(entry.getValue() - 1);
            cooldown.offer(entry);
            if (cooldown.size() < k) continue;             // ← VARIATION: wait k slots before reuse
            var ready = cooldown.poll();
            if (ready.getValue() > 0) maxHeap.offer(ready);
        }
        return builder.length() == s.length() ? builder.toString() : "";
    }
}
```

Time O(n log k) | **Space** O(k)

#480 Sliding Window Median

Description: Return the median of every window of size k as it slides across the array. **Variation:** two heaps (maxHeap = lower half, minHeap = upper half) with lazy deletion — defer removing elements that left the window, tracking balance with counts.

```java
class Solution {
    public double[] medianSlidingWindow(int[] nums, int k) {
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> Integer.compare(b, a));
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        double[] result = new double[nums.length - k + 1];
        for (int i = 0; i < nums.length; i++) {
            if (maxHeap.isEmpty() || nums[i] <= maxHeap.peek()) {
                maxHeap.offer(nums[i]);
            } else {
                minHeap.offer(nums[i]);
            }
            if (i >= k) {                                  // ← VARIATION: remove element leaving wind
                int out = nums[i - k];
                if (out <= maxHeap.peek()) {
                    maxHeap.remove(out);
                } else {
                    minHeap.remove(out);
                }
            }
            // rebalance
            while (maxHeap.size() > minHeap.size() + 1) minHeap.offer(maxHeap.poll());
            while (minHeap.size() > maxHeap.size()) maxHeap.offer(minHeap.poll());
            if (i >= k - 1)
                result[i - k + 1] = (k % 2 == 1) ? (double) maxHeap.peek()
                    : ((double) maxHeap.peek() + minHeap.peek()) / 2.0;
        }
        return result;
    }
}
```

Time O(n·k) with heap remove (O(n log k) with indexed sets) | **Space** O(k)

#692 Top K Frequent Words

Description: Return the k most frequent words. Sort by frequency descending; ties broken by lexicographic order. **Variation:** min-heap of size k ordered so the "smallest" (lowest freq, or same freq with lexicographically larger word) sits on top for eviction.

```
class Solution {
    public List<String> topKFrequent(String[] words, int k) {
        Map<String, Integer> count = new HashMap<>();
        for (String w : words) count.merge(w, 1, Integer::sum);
        PriorityQueue<String> heap = new PriorityQueue<>((a, b) ->
            count.get(a).equals(count.get(b)) ? b.compareTo(a)        // ← VARIATION: larger word evict
                                              : count.get(a) - count.get(b));
        for (String w : count.keySet()) {
            heap.offer(w);
            if (heap.size() > k) heap.poll();
        }
        List<String> result = new ArrayList<>();
        while (!heap.isEmpty()) result.add(heap.poll());
        Collections.reverse(result);
        return result;
    }
}
```

Time O(n log k) | **Space** O(n)

#772 Basic Calculator III

Description: Evaluate an arithmetic expression with `+`, `-`, `*`, `/`, and parentheses. **Variation:** combines #224 (parentheses via recursion) and #227 (operator precedence: apply `*`/`/` immediately, defer `+`/`-`).

```
class Solution {
    private int i;
    public int calculate(String s) { i = 0; return eval(s); }
    private int eval(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        int num = 0;
        char op = '+';
        while (i < s.length()) {
            char c = s.charAt(i++);
            if (Character.isDigit(c)) num = num * 10 + (c - '0');
            if (c == '(') num = eval(s);                        // ← VARIATION: recurse on '('
            if ((!Character.isDigit(c) && c != ' ') || i == s.length()) {
                if      (op == '+') stack.push(num);
                else if (op == '-') stack.push(-num);
                else if (op == '*') stack.push(stack.pop() * num);  // ← VARIATION: precedence
                else if (op == '/') stack.push(stack.pop() / num);
                op = c; num = 0;
            }
            if (c == ')') break;                                // ← VARIATION: return to caller on ')'
        }
        int sum = 0;
        while (!stack.isEmpty()) sum += stack.pop();
        return sum;
    }
}
```

Time O(n) | **Space** O(n)

Additional Reference Problems

#32 Longest Valid Parentheses

Description: Find the length of the longest valid (well-formed) parentheses substring.

Intuition: push indices onto a stack; the bottom of the stack is always the index just before the current valid run, so the gap to it gives the run length.

```
class Solution {
    public int longestValidParentheses(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(-1);                             // sentinel: index before current valid run
        int result = 0;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == '(') {
                stack.push(i);
            } else {
                stack.pop();                        // match this ')' with the top '('
                if (stack.isEmpty()) {
                    stack.push(i);                  // no match: this ')' becomes new base
                } else {
                    result = Math.max(result, i - stack.peek());
                }
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#71 Simplify Path

Description: Simplify an absolute Unix file path (handle `.`, `..`, multiple slashes).

Intuition: split on `/` and treat directories as a stack — `..` pops the last directory, `.` and empty tokens are ignored.

```java
class Solution {
    public String simplifyPath(String path) {
        Deque<String> stack = new ArrayDeque<>();
        for (String part : path.split("/")) {
            if (part.isEmpty() || part.equals(".")) {
                continue;
            } else if (part.equals("..")) {
                if (!stack.isEmpty()) stack.pop();   // go up one directory
            } else {
                stack.push(part);
            }
        }
        StringBuilder builder = new StringBuilder();
        // stack top is the deepest dir; iterate from bottom to build path in order
        Iterator<String> it = stack.descendingIterator();
        while (it.hasNext()) {
            builder.append('/').append(it.next());
        }
        return builder.length() == 0 ? "/" : builder.toString();
    }
}
```

Time O(n) | **Space** O(n)

#150 Evaluate Reverse Polish Notation

Description: Evaluate a Reverse Polish Notation expression with `+`, `-`, `*`, `/`.

Intuition: push operands; on an operator pop the two most recent operands, apply, and push the result back — exactly LIFO.

```java
class Solution {
    public int evalRPN(String[] tokens) {
        Deque<Integer> stack = new ArrayDeque<>();
        for (String token : tokens) {
            if (token.equals("+") || token.equals("-") || token.equals("*") || token.equals("/")) {
                int b = stack.pop();
                int a = stack.pop();                 // order matters for - and /
                if (token.equals("+")) {
                    stack.push(a + b);
                } else if (token.equals("-")) {
                    stack.push(a - b);
                } else if (token.equals("*")) {
                    stack.push(a * b);
                } else {
                    stack.push(a / b);
                }
            } else {
                stack.push(Integer.parseInt(token));
            }
        }
        return stack.pop();
    }
}
```

Time O(n) | **Space** O(n)

#215 Kth Largest Element in an Array

Description: Find the kth largest element in an unsorted array (not necessarily distinct).

Intuition: a min-heap of size k keeps the k largest seen so far; its top is the kth largest.

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        for (int num : nums) {
            minHeap.offer(num);
            if (minHeap.size() > k) minHeap.poll();  // evict smallest, keep k largest
        }
        return minHeap.peek();
    }
}
```

Time O(n log k) | **Space** O(k)

#218 The Skyline Problem

Description: Given building rectangles, return the skyline as a list of key points.

Intuition: sweep left to right over building edges; a max-heap of active heights tracks the tallest standing building, and a key point is emitted whenever that maximum changes.

```
class Solution {
    public List<List<Integer>> getSkyline(int[][] buildings) {
        List<int[]> events = new ArrayList<>();
        for (int[] b : buildings) {
            events.add(new int[]{b[0], -b[2]});      // start: negative height
            events.add(new int[]{b[1], b[2]});       // end:   positive height
        }
        // sort by x; at same x, starts before ends, taller start first, shorter end first
        events.sort((a, b) -> a[0] != b[0] ? a[0] - b[0] : a[1] - b[1]);
        List<List<Integer>> result = new ArrayList<>();
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
        maxHeap.offer(0);                            // ground level
        int prevMax = 0;
        for (int[] e : events) {
            if (e[1] < 0) {
                maxHeap.offer(-e[1]);                // building starts
            } else {
                maxHeap.remove(e[1]);                // building ends
            }
            int currentMax = maxHeap.peek();
            if (currentMax != prevMax) {
                result.add(Arrays.asList(e[0], currentMax));
                prevMax = currentMax;
            }
        }
        return result;
    }
}
```

Time O(n^2) (heap remove) | **Space** O(n)

#225 Implement Stack using Queues

Description: Implement a LIFO stack using only queue operations.

Intuition: after each push, rotate the queue so the newest element sits at the front — then `poll` / `peek` behave like stack `pop` / `top`.

```
class MyStack {
    Queue<Integer> queue = new ArrayDeque<>();

    public void push(int x) {
        queue.offer(x);
        for (int i = 1; i < queue.size(); i++) {     // rotate so x moves to front
            queue.offer(queue.poll());
        }
    }
    public int pop()     { return queue.poll(); }
    public int top()     { return queue.peek(); }
    public boolean empty() { return queue.isEmpty(); }
}
```

Time O(n) push, O(1) others | **Space** O(n)

#232 Implement Queue using Stacks

Description: Implement a queue using two stacks.

Intuition: one stack receives pushes, the other serves pops; moving elements over reverses their order, restoring FIFO. Amortized O(1) per element.

```
class MyQueue {
    Deque<Integer> inStack  = new ArrayDeque<>();    // ← VARIATION: two stacks emulate FIFO
    Deque<Integer> outStack = new ArrayDeque<>();

    public void push(int x) { inStack.push(x); }
    public int pop() {
        peek();                                      // ensure outStack is primed
        return outStack.pop();
    }
    public int peek() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) outStack.push(inStack.pop());  // reverse order
        }
        return outStack.peek();
    }
    public boolean empty() { return inStack.isEmpty() && outStack.isEmpty(); }
}
```

Time O(1) amortized | **Space** O(n)

#316 Remove Duplicate Letters

Description: Remove duplicate letters so each appears once, result is lexicographically smallest.

Intuition: monotone increasing stack of letters — pop a larger letter when a smaller one arrives, but only if the popped letter appears again later (so we can re-add it).

```java
class Solution {
    public String removeDuplicateLetters(String s) {
        int[] lastIndex = new int[26];
        for (int i = 0; i < s.length(); i++) lastIndex[s.charAt(i) - 'a'] = i;
        boolean[] inStack = new boolean[26];
        Deque<Character> stack = new ArrayDeque<>();
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (inStack[c - 'a']) continue;          // keep only one of each letter
            while (!stack.isEmpty() && stack.peek() > c && lastIndex[stack.peek() - 'a'] > i) {
                inStack[stack.pop() - 'a'] = false;  // pop bigger letter that recurs later
            }
            stack.push(c);
            inStack[c - 'a'] = true;
        }
        StringBuilder builder = new StringBuilder();
        Iterator<Character> it = stack.descendingIterator();
        while (it.hasNext()) builder.append(it.next());
        return builder.toString();
    }
}
```

Time O(n) | **Space** O(1) (26 letters)

#402 Remove K Digits

Description: Remove k digits from a number string to get the smallest possible number.

Intuition: monotone increasing stack of digits — whenever a smaller digit arrives, pop larger preceding digits (each pop is one removal) so high places hold the smallest digits.

```java
class Solution {
    public String removeKdigits(String num, int k) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : num.toCharArray()) {
            while (k > 0 && !stack.isEmpty() && stack.peek() > c) {
                stack.pop();                         // remove larger leading digit
                k--;
            }
            stack.push(c);
        }
        while (k > 0) {                              // still need removals: drop from top
            stack.pop();
            k--;
        }
        StringBuilder builder = new StringBuilder();
        Iterator<Character> it = stack.descendingIterator();
        while (it.hasNext()) builder.append(it.next());
        while (builder.length() > 1 && builder.charAt(0) == '0') builder.deleteCharAt(0);
        return builder.length() == 0 ? "0" : builder.toString();
    }
}
```

Time O(n) | **Space** O(n)

#456 132 Pattern

Description: Check if a 1-3-2 pattern exists in an integer array.

Variation: scan right to left with a monotone decreasing stack; pop to track the largest value that is still smaller than some element to its right (the "2"). If a later element is below that "2", a 132 pattern exists.

```
class Solution {
    public boolean find132pattern(int[] nums) {
        Deque<Integer> stack = new ArrayDeque<>();   // candidates for the "3"
        int two = Integer.MIN_VALUE;                  // ← VARIATION: best valid "2" so far
        for (int i = nums.length - 1; i >= 0; i--) {
            if (nums[i] < two) return true;           // nums[i] is the "1" < "2" < "3"
            while (!stack.isEmpty() && stack.peek() < nums[i]) {
                two = stack.pop();                    // popped value becomes the "2"
            }
            stack.push(nums[i]);
        }
        return false;
    }
}
```

Time O(n) | **Space** O(n)

#506 Relative Ranks

Description: Assign ranks ("Gold Medal", "Silver Medal", "Bronze Medal", then 4, 5, ...) to athletes by score.

Intuition: a max-heap of (score, index) pops athletes in descending score order, so each pop assigns the next rank back to its original position.

```
class Solution {
    public String[] findRelativeRanks(int[] score) {
        int n = score.length;
        PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[0] - a[0]);
        for (int i = 0; i < n; i++) maxHeap.offer(new int[]{score[i], i});
        String[] result = new String[n];
        int rank = 1;
        while (!maxHeap.isEmpty()) {
            int idx = maxHeap.poll()[1];              // highest remaining score
            if (rank == 1) {
                result[idx] = "Gold Medal";
            } else if (rank == 2) {
                result[idx] = "Silver Medal";
            } else if (rank == 3) {
                result[idx] = "Bronze Medal";
            } else {
                result[idx] = String.valueOf(rank);
            }
            rank++;
        }
        return result;
    }
}
```

Time O(n log n) | **Space** O(n)

#636 Exclusive Time of Functions

Description: Given a log of function start/end times, compute exclusive time per function.

Intuition: a call stack mirrors execution; when a new call starts, the currently running function pauses (accumulate its slice), and when a call ends, the resumed parent restarts its clock.

```java
class Solution {
    public int[] exclusiveTime(int n, List<String> logs) {
        int[] result = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();   // function ids currently running
        int prevTime = 0;
        for (String log : logs) {
            String[] parts = log.split(":");
            int id = Integer.parseInt(parts[0]);
            int time = Integer.parseInt(parts[2]);
            if (parts[1].equals("start")) {
                if (!stack.isEmpty()) result[stack.peek()] += time - prevTime;  // pause caller
                stack.push(id);
                prevTime = time;
            } else {
                result[stack.pop()] += time - prevTime + 1;  // inclusive end timestamp
                prevTime = time + 1;
            }
        }
        return result;
    }
}
```

Time O(L) (L = number of logs) | **Space** O(n)

#678 Valid Parenthesis String

Description: Check if a string with `(`, `)`, `*` (wildcard) can be a valid parenthesis string.

Variation: track a range `[low, high]` of possible open-paren counts; `*` widens the range (could be `(`, `)`, or empty). Valid iff the range can return to 0 and never goes negative.

```java
class Solution {
    public boolean checkValidString(String s) {
        int low = 0, high = 0;                      // ← VARIATION: range of open '(' counts
        for (char c : s.toCharArray()) {
            if (c == '(') {
                low++; high++;
            } else if (c == ')') {
                low--; high--;
            } else {                                // '*': treat as ')' for low, '(' for high
                low--; high++;
            }
            if (high < 0) return false;             // too many ')' even with all '*' as '('
            if (low < 0) low = 0;                   // never let open count drop below 0
        }
        return low == 0;
    }
}
```

Time O(n) | **Space** O(1)

#703 Kth Largest Element in a Stream

Description: Design a class to find the kth largest element in a stream.

Intuition: a min-heap capped at size k always holds the k largest seen; its top is the running kth largest.

```java
class KthLargest {
    private PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    private int k;

    public KthLargest(int k, int[] nums) {
        this.k = k;
        for (int num : nums) add(num);
    }
    public int add(int val) {
        minHeap.offer(val);
        if (minHeap.size() > k) minHeap.poll();      // keep only k largest
        return minHeap.peek();
    }
}
```

Time O(log k) per add | **Space** O(k)

#716 Max Stack

Description: Design a stack with push, pop, top, getMax, and popMax operations.

Variation: mirror an auxiliary `maxStack` (like #155). `popMax` pops down to the max into a temp buffer, removes it, then pushes the buffer back — re-establishing both stacks.

```java
class MaxStack {
    Deque<Integer> stack    = new ArrayDeque<>();
    Deque<Integer> maxStack = new ArrayDeque<>();    // ← VARIATION: running max alongside

    public void push(int x) {
        stack.push(x);
        maxStack.push(maxStack.isEmpty() ? x : Math.max(maxStack.peek(), x));
    }
    public int pop()  { maxStack.pop(); return stack.pop(); }
    public int top()  { return stack.peek(); }
    public int peekMax() { return maxStack.peek(); }
    public int popMax() {
        int max = maxStack.peek();
        Deque<Integer> buffer = new ArrayDeque<>();
        while (top() != max) buffer.push(pop());     // ← VARIATION: unload above the max
        pop();                                       // remove the max itself
        while (!buffer.isEmpty()) push(buffer.pop()); // reload, rebuilding maxStack
        return max;
    }
}
```

Time O(n) popMax, O(1) others | **Space** O(n)

#735 Asteroid Collision

Description: Simulate asteroids moving in a row: right-moving collide with left-moving.

Intuition: a stack holds surviving asteroids; a left-moving asteroid only collides with a right-moving one on top, resolving collisions repeatedly until it survives, explodes, or the stack clears.

```java
class Solution {
    public int[] asteroidCollision(int[] asteroids) {
        Deque<Integer> stack = new ArrayDeque<>();
        for (int a : asteroids) {
            boolean alive = true;
            while (alive && a < 0 && !stack.isEmpty() && stack.peek() > 0) {
                if (stack.peek() < -a) {
                    stack.pop();                  // top explodes, incoming continues
                } else if (stack.peek() == -a) {
                    stack.pop();                  // both explode
                    alive = false;
                } else {
                    alive = false;                // incoming explodes
                }
            }
            if (alive) stack.push(a);
        }
        int[] result = new int[stack.size()];
        for (int i = result.length - 1; i >= 0; i--) result[i] = stack.pop();
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#759 Employee Free Time

Description: Find the free time intervals common to all employees given their schedules.

Intuition: flatten all intervals, sort by start, then sweep tracking the furthest end seen so far; any gap between that end and the next start is common free time.

```java
/*
// Definition for an Interval.
class Interval {
    public int start;
    public int end;
    public Interval() {}
    public Interval(int _start, int _end) { start = _start; end = _end; }
};
*/
class Solution {
    public List<Interval> employeeFreeTime(List<List<Interval>> schedule) {
        List<Interval> all = new ArrayList<>();
        for (List<Interval> emp : schedule) all.addAll(emp);
        all.sort((a, b) -> a.start - b.start);
        List<Interval> result = new ArrayList<>();
        int prevEnd = all.get(0).end;
        for (Interval interval : all) {
            if (interval.start > prevEnd) {
                result.add(new Interval(prevEnd, interval.start));  // gap = free time
            }
            prevEnd = Math.max(prevEnd, interval.end);
        }
        return result;
    }
}
```

Time O(n log n) | **Space** O(n)

#844 Backspace String Compare

Description: Check if two strings are equal after processing `#` as backspace.

Intuition: build each string with a stack where `#` pops the last character, then compare the resulting stacks.

```
class Solution {
    public boolean backspaceCompare(String s, String t) {
        return build(s).equals(build(t));
    }
    private String build(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '#') {
                if (!stack.isEmpty()) stack.pop();   // backspace removes last char
            } else {
                stack.push(c);
            }
        }
        StringBuilder builder = new StringBuilder();
        while (!stack.isEmpty()) builder.append(stack.pop());
        return builder.toString();
    }
}
```

Time O(n) | **Space** O(n)

#856 Score of Parentheses

Description: Calculate the score of a valid parentheses string (empty=0, AB=A+B, (A)=2A or 1 if empty).

Variation: stack holds the accumulated score at each depth; push 0 on `(`, and on `)` collapse the inner score (`max(2*inner, 1)`) into the parent frame.

```
class Solution {
    public int scoreOfParentheses(String s) {
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(0);                              // ← VARIATION: score of current frame
        for (char c : s.toCharArray()) {
            if (c == '(') {
                stack.push(0);                      // new inner frame
            } else {
                int inner = stack.pop();
                int add = inner == 0 ? 1 : 2 * inner;  // "()"=1, "(A)"=2A
                stack.push(stack.pop() + add);      // fold into parent
            }
        }
        return stack.pop();
    }
}
```

Time O(n) | **Space** O(n)

#862 Shortest Subarray with Sum at Least K

Description: Find the length of the shortest subarray with sum at least k (k can be negative).

Variation: monotone increasing deque over prefix sums — pop from the front when a window qualifies, and from the back when a newer prefix is smaller (dominating older larger ones).

```java
class Solution {
    public int shortestSubarray(int[] nums, int k) {
        int n = nums.length;
        long[] prefix = new long[n + 1];
        for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] + nums[i];
        Deque<Integer> queue = new ArrayDeque<>();   // indices, prefix increasing front→back
        int result = n + 1;
        for (int i = 0; i <= n; i++) {
            while (!queue.isEmpty() && prefix[i] - prefix[queue.peekFirst()] >= k) {
                result = Math.min(result, i - queue.pollFirst());  // ← VARIATION: window qualifies
            }
            while (!queue.isEmpty() && prefix[queue.peekLast()] >= prefix[i]) {
                queue.pollLast();                    // ← VARIATION: drop dominated prefixes
            }
            queue.offerLast(i);
        }
        return result <= n ? result : -1;
    }
}
```

Time O(n) | **Space** O(n)

#907 Sum of Subarray Minimums

Description: Find the sum of subarray minimums for all subarrays.

Variation: monotone increasing stack — for each element as the minimum, count subarrays via distance to the previous strictly-smaller and next smaller-or-equal element, then weight by the value.

```java
class Solution {
    public int sumSubarrayMins(int[] arr) {
        int n = arr.length;
        long MOD = 1_000_000_007L;
        Deque<Integer> stack = new ArrayDeque<>();   // indices, values increasing bottom→top
        long sum = 0;
        for (int i = 0; i <= n; i++) {
            int current = i == n ? Integer.MIN_VALUE : arr[i];  // sentinel flushes stack
            while (!stack.isEmpty() && arr[stack.peek()] > current) {
                int mid = stack.pop();               // arr[mid] is the subarray minimum
                int left = stack.isEmpty() ? -1 : stack.peek();
                long count = (long)(mid - left) * (i - mid);  // ← VARIATION: subarrays where mid is mi
                sum = (sum + count * arr[mid]) % MOD;
            }
            stack.push(i);
        }
        return (int) sum;
    }
}
```

Time O(n) | **Space** O(n)

#921 Minimum Add to Make Parentheses Valid

Description: Find the minimum additions to make a parentheses string valid.

Intuition: track open parentheses as a counter (stack of size); each unmatched `)` needs an added `(`, and leftover `(` each need a `)`.

```java
class Solution {
    public int minAddToMakeValid(String s) {
        int open = 0;                               // unmatched '(' (stack height)
        int additions = 0;
        for (char c : s.toCharArray()) {
            if (c == '(') {
                open++;
            } else {
                if (open > 0) {
                    open--;                         // match an existing '('
                } else {
                    additions++;                    // need to add a '(' for this ')'
                }
            }
        }
        return additions + open;                    // remaining '(' each need a ')'
    }
}
```

Time O(n) | **Space** O(1)

#946 Validate Stack Sequences

Description: Check if a sequence can be produced by a series of push-pop operations on a stack.

Intuition: simulate — push each value, then greedily pop whenever the stack top matches the next expected popped value. If everything pops, the sequence is valid.

```java
class Solution {
    public boolean validateStackSequences(int[] pushed, int[] popped) {
        Deque<Integer> stack = new ArrayDeque<>();
        int j = 0;                                  // index into popped
        for (int value : pushed) {
            stack.push(value);
            while (!stack.isEmpty() && stack.peek() == popped[j]) {
                stack.pop();                        // pop as long as top matches
                j++;
            }
        }
        return stack.isEmpty();
    }
}
```

Time O(n) | **Space** O(n)

#973 K Closest Points to Origin

Description: Find the k points closest to the origin.

Intuition: a max-heap of size k keyed on squared distance keeps the k closest seen; evict the farthest whenever the heap overflows.

```java
class Solution {
    public int[][] kClosest(int[][] points, int k) {
        PriorityQueue<int[]> maxHeap = new PriorityQueue<>(
            (a, b) -> (b[0] * b[0] + b[1] * b[1]) - (a[0] * a[0] + a[1] * a[1]));
        for (int[] point : points) {
            maxHeap.offer(point);
            if (maxHeap.size() > k) maxHeap.poll();  // evict farthest, keep k closest
        }
        int[][] result = new int[k][2];
        for (int i = 0; i < k; i++) result[i] = maxHeap.poll();
        return result;
    }
}
```

Time O(n log k) | **Space** O(k)

#1086 High Five

Description: For each student, compute the average of their top five scores.

Intuition: keep a min-heap of size 5 per student so only their five highest scores remain, then average those.

```java
class Solution {
    public int[][] highFive(int[][] items) {
        Map<Integer, PriorityQueue<Integer>> map = new TreeMap<>();  // sorted by student id
        for (int[] item : items) {
            int id = item[0], score = item[1];
            PriorityQueue<Integer> minHeap = map.computeIfAbsent(id, key -> new PriorityQueue<>());
            minHeap.offer(score);
            if (minHeap.size() > 5) minHeap.poll();  // keep top 5 scores
        }
        int[][] result = new int[map.size()][2];
        int i = 0;
        for (Map.Entry<Integer, PriorityQueue<Integer>> e : map.entrySet()) {
            int sum = 0;
            for (int score : e.getValue()) sum += score;
            result[i][0] = e.getKey();
            result[i][1] = sum / 5;
            i++;
        }
        return result;
    }
}
```

Time O(n log 5) | **Space** O(n)

#1190 Reverse Substrings Between Each Pair of Parentheses

Description: Reverse the substrings between each pair of parentheses (innermost first).

Variation: stack of builders — push current builder on `(`; on `)` reverse the inner builder and append it to the parent frame.

```
class Solution {
    public String reverseParentheses(String s) {
        Deque<StringBuilder> stack = new ArrayDeque<>();  // ← VARIATION: builder per frame
        StringBuilder current = new StringBuilder();
        for (char c : s.toCharArray()) {
            if (c == '(') {
                stack.push(current);                   // save parent frame
                current = new StringBuilder();
            } else if (c == ')') {
                current.reverse();                     // ← VARIATION: reverse inner substring
                stack.peek().append(current);
                current = stack.pop();
            } else {
                current.append(c);
            }
        }
        return current.toString();
    }
}
```

Time O(n^2) | **Space** O(n)

#1209 Remove All Adjacent Duplicates in String II

Description: Remove all adjacent duplicates of length k in a string repeatedly.

Variation: stack of (char, count) pairs — increment the top's count on a repeat, and pop the frame once its count reaches k.

```
class Solution {
    public String removeDuplicates(String s, int k) {
        Deque<int[]> stack = new ArrayDeque<>();     // ← VARIATION: [charCode, runLength]
        for (char c : s.toCharArray()) {
            if (!stack.isEmpty() && stack.peek()[0] == c) {
                stack.peek()[1]++;
                if (stack.peek()[1] == k) stack.pop();  // full group of k: remove
            } else {
                stack.push(new int[]{c, 1});
            }
        }
        StringBuilder builder = new StringBuilder();
        Iterator<int[]> it = stack.descendingIterator();
        while (it.hasNext()) {
            int[] frame = it.next();
            for (int i = 0; i < frame[1]; i++) builder.append((char) frame[0]);
        }
        return builder.toString();
    }
}
```

Time O(n) | **Space** O(n)

#1249 Minimum Remove to Make Valid Parentheses

Description: Remove the minimum number of parentheses to make the string valid.

Variation: stack stores indices of unmatched `(`; a `)` with no match is marked for removal, and any `(` left on the stack at the end is also removed.

```java
class Solution {
    public String minRemoveToMakeValid(String s) {
        char[] chars = s.toCharArray();
        Deque<Integer> stack = new ArrayDeque<>();   // ← VARIATION: indices of unmatched '('
        for (int i = 0; i < chars.length; i++) {
            if (chars[i] == '(') {
                stack.push(i);
            } else if (chars[i] == ')') {
                if (stack.isEmpty()) {
                    chars[i] = '*';                  // unmatched ')': mark for removal
                } else {
                    stack.pop();
                }
            }
        }
        while (!stack.isEmpty()) chars[stack.pop()] = '*';  // unmatched '('
        StringBuilder builder = new StringBuilder();
        for (char c : chars) {
            if (c != '*') builder.append(c);
        }
        return builder.toString();
    }
}
```

Time O(n) | **Space** O(n)

#1337 The K Weakest Rows in a Matrix

Description: Find the k weakest rows in a matrix (fewest soldiers, ties broken by row index).

Variation: max-heap of size k keyed on (soldier count, row index) so the strongest qualifying row sits on top for eviction; the remaining k are the weakest.

```java
class Solution {
    public int[] kWeakestRows(int[][] mat, int k) {
        // max-heap: strongest row (more soldiers, or larger index on tie) on top
        PriorityQueue<int[]> maxHeap = new PriorityQueue<>(
            (a, b) -> a[0] != b[0] ? b[0] - a[0] : b[1] - a[1]);
        for (int i = 0; i < mat.length; i++) {
            int soldiers = 0;
            for (int v : mat[i]) soldiers += v;
            maxHeap.offer(new int[]{soldiers, i});   // ← VARIATION: (count, index)
            if (maxHeap.size() > k) maxHeap.poll();  // evict strongest
        }
        int[] result = new int[k];
        for (int i = k - 1; i >= 0; i--) result[i] = maxHeap.poll()[1];
        return result;
    }
}
```

Time O(m·n + m log k) | **Space** O(k)

#1541 Minimum Insertions to Balance a Parentheses String

Description: Find the minimum insertions to make a string balanced (every `(` needs two `)`).

Variation: counter-style stack tracking open `(`; each `(` demands two `)`. Handle `)` carefully since they come in pairs, inserting a `)` when only one is available.

```java
class Solution {
    public int minInsertions(String s) {
        int open = 0;                               // unmatched '(' needing ')) '
        int insertions = 0;
        int i = 0;
        while (i < s.length()) {
            if (s.charAt(i) == '(') {
                open++;
                i++;
            } else {                                // ')': need two in a row
                if (i + 1 < s.length() && s.charAt(i + 1) == ')') {
                    i += 2;
                } else {
                    insertions++;                   // ← VARIATION: insert missing ')'
                    i++;
                }
                if (open > 0) {
                    open--;
                } else {
                    insertions++;                   // ← VARIATION: insert missing '('
                }
            }
        }
        return insertions + open * 2;               // each leftover '(' needs '))'
    }
}
```

Time O(n) | **Space** O(1)

#1614 Maximum Nesting Depth of the Parentheses

Description: Find the maximum nesting depth of parentheses in a string.

Intuition: the stack depth equals the current nesting level; track a running counter for `(`/`)` and record its peak.

```java
class Solution {
    public int maxDepth(String s) {
        int depth = 0;                              // current stack height
        int result = 0;
        for (char c : s.toCharArray()) {
            if (c == '(') {
                depth++;
                result = Math.max(result, depth);
            } else if (c == ')') {
                depth--;
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#1792 Maximum Average Pass Ratio

Description: Maximize the average pass ratio by adding extra students optimally.

Variation: max-heap keyed on the marginal gain from adding one student to a class; greedily assign each extra student where it helps most, then push the updated gain back.

```java
class Solution {
    public double maxAverageRatio(int[][] classes, int extraStudents) {
        // max-heap by gain from adding one passing student
        PriorityQueue<double[]> maxHeap = new PriorityQueue<>(
            (a, b) -> Double.compare(b[2], a[2]));   // ← VARIATION: order by marginal gain
        for (int[] c : classes) {
            double pass = c[0], total = c[1];
            maxHeap.offer(new double[]{pass, total, gain(pass, total)});
        }
        while (extraStudents-- > 0) {
            double[] top = maxHeap.poll();
            double pass = top[0] + 1, total = top[1] + 1;  // add one passing student
            maxHeap.offer(new double[]{pass, total, gain(pass, total)});
        }
        double sum = 0;
        for (double[] c : maxHeap) sum += c[0] / c[1];
        return sum / classes.length;
    }
    private double gain(double pass, double total) {
        return (pass + 1) / (total + 1) - pass / total;
    }
}
```

Time O((n + extra) log n) | **Space** O(n)

#1944 Number of Visible People in a Queue

Description: Count the number of people each person can see in a queue (taller blocks view).

Variation: monotone decreasing stack scanned right to left; pop shorter people (each is visible) until a taller one blocks the view — that taller person is visible too.

```java
class Solution {
    public int[] canSeePersonsCount(int[] heights) {
        int n = heights.length;
        int[] result = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();   // heights, decreasing bottom→top
        for (int i = n - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() < heights[i]) {
                stack.pop();                         // ← VARIATION: shorter person is visible
                result[i]++;
            }
            if (!stack.isEmpty()) result[i]++;       // the taller blocker is also visible
            stack.push(heights[i]);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#1985 Find the Kth Largest Integer in the Array

Description: Find the kth largest integer in an array of number strings.

Variation: min-heap of size k comparing the numeric strings by length first, then lexicographically (so big-integer values compare correctly without overflow).

```java
class Solution {
    public String kthLargestNumber(String[] nums, int k) {
        // ← VARIATION: compare by length then lexicographically (numeric order for big ints)
        PriorityQueue<String> minHeap = new PriorityQueue<>(
            (a, b) -> a.length() != b.length() ? a.length() - b.length() : a.compareTo(b));
        for (String num : nums) {
            minHeap.offer(num);
            if (minHeap.size() > k) minHeap.poll();  // keep k largest
        }
        return minHeap.peek();
    }
}
```

Time O(n log k) | **Space** O(k)